

Universidad Veracruzana - Región Xalapa

# Proyecto - Página web de catálogo de servicios tecnológicos

Desarrollo de Sistemas Web

Abraham Núñez Sánchez  
Turan Ozbek  
Rafael Alejandro Díaz Rangel  
Alan Penagos Gonzales

Junio 2026

# Indice

|                                          |    |
|------------------------------------------|----|
| Indice.....                              | 1  |
| 1. Introducción.....                     | 3  |
| 2. Información general del proyecto..... | 3  |
| 3. Planteamiento del problema.....       | 4  |
| 4. Objetivo general.....                 | 5  |
| 5. Descripción de la solución.....       | 5  |
| 5.1. Nombre del proyecto.....            | 5  |
| 5.2. Problemática a resolver.....        | 5  |
| 5.3. Objetivo general del sistema.....   | 5  |
| 5.4. Público objetivo.....               | 6  |
| 6. User Persona.....                     | 6  |
| 6.1. Información del usuario.....        | 6  |
| 6.2. Objetivos principales.....          | 7  |
| 6.3. Necesidades.....                    | 7  |
| 6.4. Problemas actuales.....             | 7  |
| 7. Requisitos funcionales.....           | 8  |
| 8. Arquitectura de la información.....   | 9  |
| 8.1. Descripción.....                    | 9  |
| 8.2. Estructura general del sitio.....   | 10 |
| 9. Mapa del sitio.....                   | 12 |
| 10. Menu de navegación.....              | 13 |
| 11. Wireframes.....                      | 14 |
| 11.1. Página Principal.....              | 15 |
| 11.2. Página “Nosotros”.....             | 16 |
| 11.3. Servicios.....                     | 17 |
| 11.3.1. Formulario de solicitud.....     | 18 |
| 11.4. Estadísticas - Dashboard.....      | 19 |
| 11.5. Contacto.....                      | 20 |
| 12. Paleta de colores y tipografía.....  | 23 |
| 13. Modelo relacional.....               | 24 |
| Tabla 1 — categorías.....                | 26 |
| Tabla 2 — servicios.....                 | 27 |
| Tabla 3 — solicitudes.....               | 28 |
| 14. Tecnologías empleadas.....           | 29 |
| 14.1. Frontend.....                      | 30 |
| 14.2. Backend.....                       | 31 |
| 14.3. Base de datos.....                 | 31 |

|                                                                      |    |
|----------------------------------------------------------------------|----|
| 14.4. Comunicación dinámica.....                                     | 32 |
| 15. Diagrama del stack tecnológico.....                              | 32 |
| 16. Estructura MVC.....                                              | 33 |
| 17. Explicación MVC.....                                             | 34 |
| 18. Módulos implementados.....                                       | 36 |
| 19. Mecanismos de seguridad.....                                     | 37 |
| 19.1. Prevención de Inyección SQL (Consultas Parametrizadas).....    | 37 |
| 19.2. Prevención de Cross-Site Scripting (XSS).....                  | 38 |
| 19.3. Validación de Datos de Entrada.....                            | 38 |
| 19.4. Validación y Restricción de Archivos Subidos.....              | 39 |
| 19.5. Gestión de Variables de Entorno (Credenciales).....            | 40 |
| 19.6. Manejo Centralizado de Errores.....                            | 41 |
| 19.7. Política de Acceso CORS.....                                   | 41 |
| 19.8. Integridad Referencial en la Base de Datos.....                | 41 |
| 19.9. Áreas de Mejora Identificadas.....                             | 42 |
| 20. Modelado de amenaza.....                                         | 42 |
| 21. Evidencias de pruebas.....                                       | 45 |
| 21.1. Auditoría con Google Lighthouse.....                           | 45 |
| 21.2. Resultados por Categoría.....                                  | 46 |
| 21.3. Interpretación de las Métricas de Rendimiento.....             | 47 |
| 21.4. Prueba de solicitud.....                                       | 47 |
| 21.5. Prueba de solicitud errónea.....                               | 50 |
| 21.6. Email falso.....                                               | 52 |
| 21.7. Pruebas de Inyección de SQL y Cross-Site Scripting (XSS).....  | 52 |
| 21.8. Página Web responsiva.....                                     | 54 |
| 22. Problemas encontrados y soluciones.....                          | 57 |
| 22.1. Duplicación completa del código en main.js.....                | 57 |
| 22.2. Ausencia de compresión en las respuestas del servidor.....     | 58 |
| 22.3. Recursos que bloquean el renderizado inicial.....              | 58 |
| 22.4. CSS de Bootstrap mayormente sin usar.....                      | 59 |
| 22.5. Cambio de diseño (layout shift) al cargar la fuente Inter..... | 59 |
| 22.6. JavaScript sin minificar.....                                  | 60 |
| 23. Reflexión final.....                                             | 60 |
| 24. Conclusiones.....                                                | 61 |
| 25. Referencias APA.....                                             | 62 |

# 1. Introducción

Actualmente, las empresas y organizaciones dependen cada vez más de soluciones tecnológicas que permitan optimizar sus procesos, mejorar la comunicación con sus clientes y proteger la información que manejan. El crecimiento de los servicios digitales ha generado la necesidad de desarrollar plataformas web que sean accesibles, funcionales y que integren principios de seguridad desde su diseño. El presente proyecto consiste en el desarrollo de una plataforma web para SeguriTech, un catálogo digital de servicios tecnológicos orientado a la presentación de soluciones relacionadas con infraestructura, desarrollo web y ciberseguridad. La aplicación permite a los usuarios consultar los servicios disponibles, conocer información general de la empresa, enviar solicitudes de cotización y facilitar la gestión de la información mediante un sistema estructurado.

Para el desarrollo de la solución se empleó una arquitectura basada en el patrón Modelo-Vista-Controlador (MVC), permitiendo separar la lógica de negocio, manejo de datos e interfaz de usuario. La implementación utiliza tecnologías web como Node.js, Express.js, EJS, Bootstrap, JavaScript, CSS y una base de datos relacional mediante MySQL, logrando una estructura organizada, escalable y mantenible. Además, se consideraron aspectos relacionados con seguridad informática, incluyendo validación de formularios, manejo de archivos, separación de componentes del sistema y un modelo básico de amenazas para identificar posibles riesgos asociados a la aplicación web.

Este documento presenta la planificación, diseño e implementación del proyecto, describiendo los objetivos, requerimientos funcionales, arquitectura de información, diseño visual, modelo de datos, arquitectura tecnológica, estructura MVC, mecanismos de seguridad aplicados, pruebas realizadas y las conclusiones obtenidas durante el desarrollo de la solución.

## 2. Información general del proyecto

Cada equipo desarrollará un sistema web sobre una temática elegida, que incluirá funcionalidades de registro y consulta de información. La aplicación deberá contar con:

- Interfaz web responsiva.

- Formularios de captura de información.
- Validación en cliente y servidor.
- Carga dinámica de datos mediante AJAX.
- Persistencia en MySQL.
- Arquitectura MVC.
- Subida de archivos.
- Dashboard o sección de indicadores.
- Seguridad

### 3. Planteamiento del problema

En la actualidad, las organizaciones dependen de aplicaciones web para automatizar procesos, ofrecer servicios en línea y gestionar información de manera eficiente. Sitios como sistemas de soporte técnico, plataformas de comercio electrónico, portales académicos y sistemas de reservaciones integran múltiples tecnologías para proporcionar experiencias dinámicas e interactivas a los usuarios. Detrás de estas aplicaciones existe una arquitectura que separa claramente la interfaz de usuario, la lógica del negocio y el acceso a los datos. Esta separación, implementada comúnmente mediante el patrón de diseño Modelo–Vista–Controlador (MVC), permite desarrollar sistemas más mantenibles, escalables y reutilizables.

En este contexto, se plantea el desarrollo de una aplicación web dinámica completa que integre:

- Estructura semántica con HTML5.
- Presentación visual responsiva con CSS3.
- Interactividad del lado del cliente con JavaScript.
- Comunicación asíncrona con AJAX y fetch().
- Persistencia de datos en MySQL.

- Backend con Node.js y Express.
- Motor de vistas EJS.
- Arquitectura MVC.

El proyecto permitirá a los estudiantes aplicar de forma integradora los conocimientos adquiridos durante el curso, desarrollando un sistema funcional similar a los utilizados en entornos profesionales.

## 4. Objetivo general

Diseñar, desarrollar e implementar una aplicación web dinámica que resuelva una problemática específica, empleando tecnologías frontend y backend modernas, base de datos relacional y arquitectura MVC.

## 5. Descripción de la solución

### 5.1. Nombre del proyecto

SeguriTech - Empresa de Servicios Tecnológicos (Desarrollo de Software, Ciberseguridad, Redes de Computo).

### 5.2. Problemática a resolver

Somos una empresa de tecnología enfocada en el desarrollo de software, redes y servicios de ciberseguridad que tiene múltiples servicios para diversos clientes de acuerdo con las necesidades individuales de cada negocio, empresa y cliente. Por lo cual se tiene un portal web, en donde los servicios ofrecidos por SeguriTech, puedan ser ofrecidos de la mejor manera, y hacer lo más accesible posible.

### 5.3. Objetivo general del sistema

- Mostrar a los clientes los servicios disponibles de la manera más accesible posible e implementar un portal web, que pueda cumplir con las necesidades del negocio. Poder ofrecer servicios de:

- Implementación de Redes de Computo.
- Mantenimiento de Redes de Computo.
- Implementación de Seguridad en la Redes de Computo.
- Automatización de Redes y Servicios de Cómputo.
- Desarrollo de Software de Ciberseguridad (Scripts en Python, Bash, Herramientas de monitoreo).
- Auditorías de ciberseguridad.
- Pentesting (Web, Red, Infraestructura).
- Administración de Servidores (Linux, RedHat, Debian y Ubuntu).
- Configuración de Firewalls.
- Capacitación de Personal de Ciberseguridad.

#### 5.4. Público objetivo

- Directores de Tecnología (CTO) e IT.
- Fundadores y Directores Generales (CEO/Propietarios) de PyMEs.
- Gerentes de Cumplimiento (Compliance/Legal).
- Directores de Seguridad de la información (CISO).
- Profesionales en el área de TI
- Departamentos de TI asociados a empresas
- Usuarios interesados en la ciberseguridad
- Ejecutivos interesados en adquirir servicios de software y ciberseguridad.

## 6. User Persona

### 6.1. Información del usuario

- Nombre: Sebastian Casique Saldaña
- Jefe del departamento de tecnología en una empresa de manufacturación mediana.
- Perfil: Sebastián es responsable de garantizar la disponibilidad, seguridad y correcto funcionamiento de los sistemas tecnológicos utilizados por la empresa. Debido al crecimiento de la organización, busca proveedores especializados que puedan ayudarlo a fortalecer la infraestructura tecnológica, reducir riesgos de seguridad y mejorar la administración de sus recursos digitales.

- Nivel técnico: Alto, conoce arquitecturas, redes y seguridad empresarial. Tiene experiencia técnica suficiente para evaluar soluciones, comparar propuestas y entender conceptos relacionados con ciberseguridad, redes y servidores, por lo que requiere información clara, técnica y confiable antes de contratar un servicio.
- Meta principal: Modernizar la infraestructura tecnológica de su empresa y garantizar la seguridad de los sistemas críticos
- Servicios de interés: Auditorías de ciberseguridad, Pentesting, Administración de Servidores, Configuración de Firewalls

## 6.2. Objetivos principales

- Modernizar la infraestructura tecnológica de la empresa.
- Mejorar la seguridad de los sistemas críticos.
- Reducir vulnerabilidades y riesgos asociados a ataques informáticos.
- Garantizar la continuidad operativa de los servicios tecnológicos.
- Encontrar proveedores especializados en soluciones empresariales.

## 6.3. Necesidades

- Contar con servicios profesionales de ciberseguridad.
- Obtener diagnósticos técnicos sobre la infraestructura actual.
- Implementar controles de seguridad como firewalls, monitoreo y auditorías.
- Recibir asesoría personalizada según las necesidades de la empresa.
- Tener un canal sencillo para solicitar cotizaciones y contactar especialistas.

## 6.4. Problemas actuales

- Dificultad para encontrar proveedores confiables con experiencia empresarial.

SeguriTech permite a Sebastián consultar servicios tecnológicos especializados, conocer las soluciones disponibles y enviar solicitudes de cotización de forma estructurada, facilitando el contacto con especialistas capaces de atender necesidades de infraestructura y seguridad empresarial.



## 7. Requisitos funcionales

| ID   | Requisito funcional                 | Descripción                                                                                                                                                                                                                   |
|------|-------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RF-1 | Página principal informativa        | El sistema deberá mostrar una página de inicio donde se presente la empresa SeguriTech.                                                                                                                                       |
| RF-2 | Información institucional           | El sistema deberá permitir visualizar información relacionada con la empresa, incluyendo misión, visión, objetivos, filosofía empresarial y presentación del equipo creativo o integrantes del proyecto.                      |
| RF-3 | Catálogo de servicios               | El sistema deberá mostrar el catálogo de servicios tecnológicos ofrecidos por SeguriTech, obteniendo la información almacenada en la base de datos y presentándola mediante una interfaz accesible y organizada.              |
| RF-4 | Filtrado de servicios por categoría | El sistema deberá permitir al usuario consultar servicios específicos mediante filtros de categoría, facilitando la búsqueda de soluciones relacionadas con ciberseguridad, infraestructura, desarrollo web u otras áreas.    |
| RF-5 | Consulta dinámica de información    | El sistema deberá cargar información de servicios mediante solicitudes asíncronas (AJAX), evitando recargas completas de página y mejorando la experiencia del usuario.                                                       |
| RF-6 | Solicitud de cotización             | El sistema deberá permitir que los usuarios envíen solicitudes de cotización mediante un formulario de contacto, incluyendo datos personales, servicio requerido, descripción de la necesidad y archivos adjuntos opcionales. |
| RF-7 | Validación de formularios           | El sistema deberá validar la información ingresada en los formularios tanto del lado del cliente como del servidor, verificando campos obligatorios, formatos correctos y restricciones de datos.                             |

|       |                                        |                                                                                                                                                                                                                     |
|-------|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RF-8  | Registro de solicitudes                | El sistema deberá almacenar las solicitudes de servicio enviadas por los usuarios dentro de la base de datos, asociándolas con el servicio seleccionado y conservando la información necesaria para su seguimiento. |
| RF-9  | Panel estadístico                      | El sistema deberá mostrar un dashboard administrativo con indicadores sobre solicitudes registradas, servicios más solicitados, cantidad de solicitudes por servicio y estados de atención.                         |
| RF-10 | Visualización dinámica de estadísticas | El sistema deberá obtener y actualizar la información estadística mediante consultas al servidor y mostrarla dinámicamente mediante componentes visuales.                                                           |
| RF-11 | Navegación del sitio                   | El sistema deberá proporcionar una barra de navegación que permita acceder a las secciones principales: Inicio, Nosotros, Servicios, Estadísticas y Contacto.                                                       |
| RF-12 | Diseño adaptable                       | El sistema deberá mostrar una interfaz adaptable a diferentes tamaños de pantalla mediante tecnologías de diseño responsivo.                                                                                        |

## 8. Arquitectura de la información

### 8.1. Descripción

SeguriTech está construido sobre la arquitectura MVC (Modelo–Vista–Controlador) implementada con Node.js y Express. Esta arquitectura separa claramente tres responsabilidades: los Modelos gestionan el acceso a la base de datos MySQL, los Controladores contienen la lógica de negocio y procesan las peticiones HTTP, y las Vistas renderizan la interfaz de usuario mediante el motor de plantillas EJS.

La arquitectura de información del sistema web SeguriTech se diseñó con el objetivo de organizar el contenido de manera clara, accesible y orientada al usuario. La estructura permite que los visitantes puedan conocer la empresa, consultar los servicios disponibles y realizar solicitudes de cotización mediante una navegación sencilla.

El sitio se organiza mediante una estructura jerárquica donde la información principal se encuentra accesible desde la barra de navegación principal.

## 8.2. Estructura general del sitio

La plataforma está dividida en las siguientes secciones principales:

### - Inicio

**Objetivo:** Presentar la identidad de SeguriTech y proporcionar una primera vista general de los servicios ofrecidos.

Contenido:

- Nombre y descripción de la empresa.
- Propuesta de valor.
- Áreas principales de servicio.
- Accesos rápidos hacia servicios y contacto.

### - Nosotros

**Objetivo:** Brindar información institucional para generar confianza en los usuarios.

Contenido:

- Descripción de la empresa.
- Misión.

- Visión.
- Equipo creativo o integrantes del proyecto.
- Servicios

**Objetivo:** Mostrar el catálogo de soluciones tecnológicas disponibles.

Contenido:

- Lista de servicios registrados en la base de datos.
- Clasificación por categorías.
- Descripción de cada servicio.
- Opción para solicitar información o cotización.

Ejemplo de categorías:

- Ciberseguridad.
- Infraestructura tecnológica.
- Desarrollo web.
- Administración de sistemas.
- **Contacto**

**Objetivo:** Facilitar la comunicación entre el usuario y SeguriTech.

Contenido:

- Información de contacto.
- Teléfono.
- Correo electrónico.
- Ubicación.

- Formulario de solicitud de cotización.

El formulario permite capturar:

- Nombre del usuario.
- Correo.
- Teléfono.
- Servicio requerido.
- Mensaje.
- Archivo adjunto opcional.
- Dashboard / Estadísticas

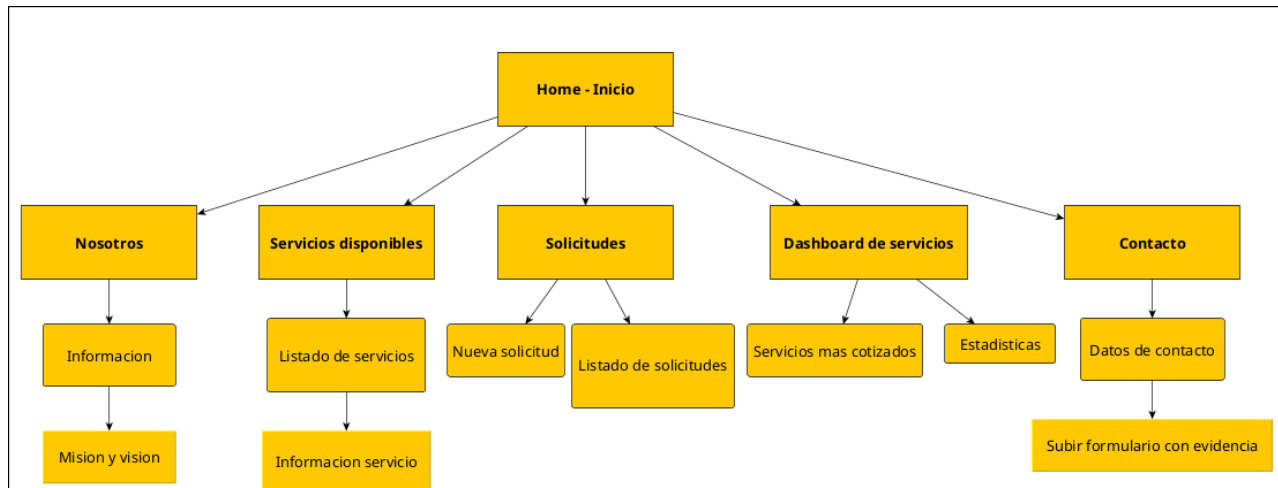
**Objetivo:** Mostrar información administrativa generada a partir de los datos almacenados.

Contenido:

- Total de solicitudes.
- Servicios más solicitados.
- Distribución de solicitudes.
- Indicadores generales del sistema.

## 9. Mapa del sitio

El portal de SeguriTech tiene 5 secciones principales accesibles desde el menú de navegación:



- Inicio: Es la raíz del sitio con el banner principal, resumen de la empresa y acceso rápido a servicios.
- Nosotros: Contiene "Quiénes somos" y "Misión y visión", para generar confianza en los visitantes antes de contratar.
- Servicios: Es el módulo central. Permite ver el catálogo completo y filtrar por categoría (Redes, Ciberseguridad, Desarrollo de Software). Cada servicio tiene su propia vista de detalle.
- Solicitudes: Es el módulo funcional principal del sistema: permite registrar una nueva solicitud/cotización (con subida de archivo adjunto) y listar/consultar las solicitudes existentes.
- Dashboard: Muestra estadísticas dinámicas cargadas vía AJAX: servicios más consultados, solicitudes por categoría, totales, etc.
- Contacto: Incluye el formulario de contacto general con subida de evidencia opcional.

Nota: Este mapeado es fue una idea que fue cambiando durante la realizacion del proyecto, sin embargo se intento mantener el proyecto lo mas fiel posible al mapeo original.

## 10. Menu de navegacion



[Logo SeguriTech]    Nosotros | Catalogo (Servicios) | Solicitudes | Estadisticas (Dashboard) |  
Contacto

El menú es fijo (sticky), responsivo y colapsa en un ícono hamburguesa en pantallas pequeñas.  
La sección activa se resalta visualmente.

Nota, durante el desarrollo del proyecto se modificó la sección de “Solicitar”, moviendo el formulario principal a la sección de “Servicios”. Quedando de la siguiente manera.

[Logo SeguriTech]    Inicio | Nosotros | Catalogo (Servicios) | Estadisticas (Dashboard) | Contacto

## 11. Wireframes

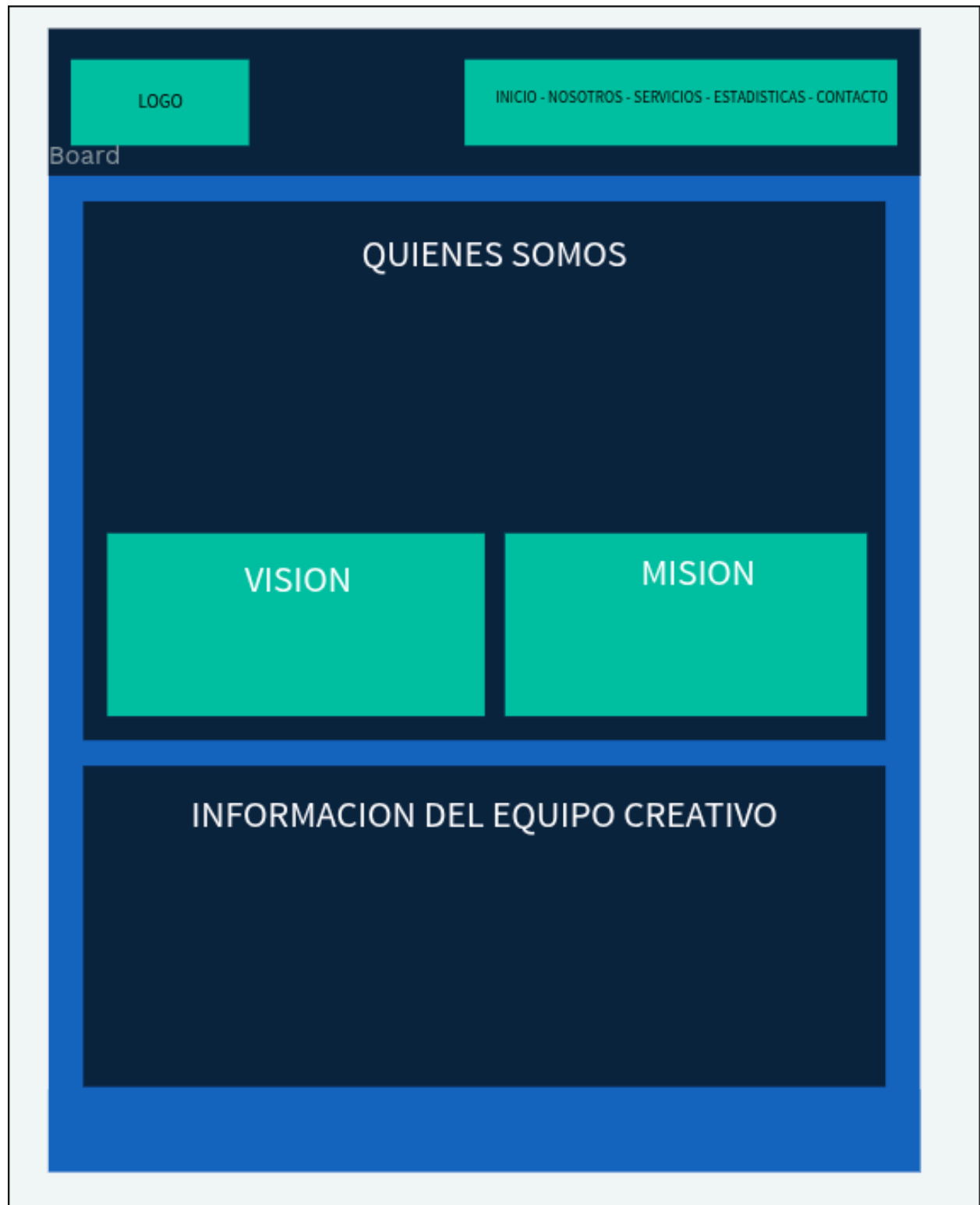
Los wireframes fueron creados mediante la herramienta “Penpot” es una aplicación web colaborativa de código abierto para el diseño de interfaces. Está desarrollado por Kaleidos, empresa española con sede en Madrid.

## 11.1. Pagina Principal





## 11.2. Pagina “Nosotros”



### 11.3. Servicios



### 11.3.1. Formulario de solicitud

#### Nueva solicitud de servicio

DATOS DEL SOLICITANTE

Nombre completo \*

Ej. Roberto Castillo

Empresa \*

Nombre de la empresa

Correo electrónico \*

correo@empresa.com

Teléfono

10 dígitos

Validación de formato en cliente y servidor

Solo números, 10 caracteres

DETALLES DEL SERVICIO

Categoría de servicio \*

— Selecciona una categoría —

Servicio específico \*

— Selecciona primero una categoría (carga vía AJAX) —

Se carga dinámicamente con fetch() según la categoría seleccionada

Descripción de la necesidad \*

Describe brevemente lo que necesitas...

Mínimo 20 caracteres, máximo 1000

ARCHIVO ADJUNTO

Documento / Evidencia

Arrastra un archivo o haz clic para seleccionar

PDF, JPG, PNG — Máx. 5 MB

Validación: extensión, tipo MIME y tamaño en servidor

Los campos marcados con \* son obligatorios. Los datos serán validados en cliente (JS) y servidor (Node.js/Express).

Cancelar

Enviar solicitud

## 11.4. Estadísticas - Dashboard

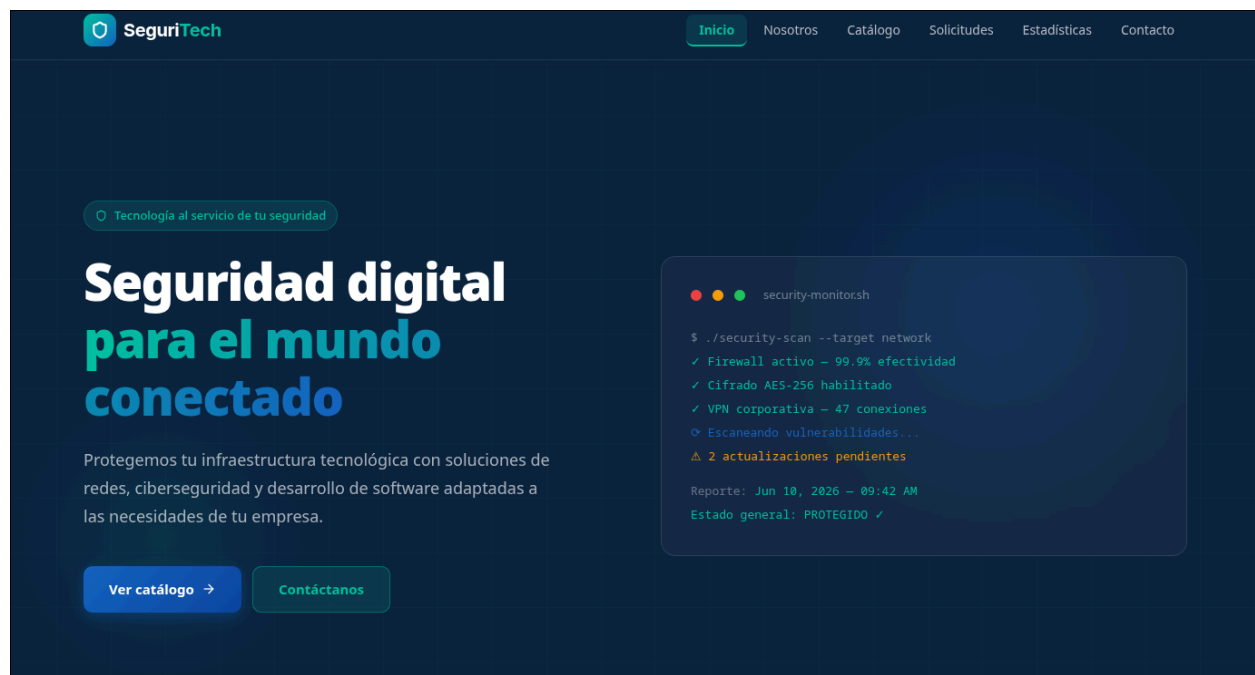


## 11.5. Contacto



Posterior a la creacion los wireframes de manera manual y definir la estructura inicial, se uso la herramienta llamada “Figma” que es un editor de gráficos vectorial y una herramienta de generación de prototipos, principalmente basada en la web, con características off-line adicionales habilitadas por aplicaciones de escritorio en macOS y Windows. Figma ofrece la oportunidad de “renderizar” un wireframe mediante una descripcion del mismo para mediante inteligencia artificial crear un prototipo de como se veria ya terminado. Se considero usar algunos elementos vistos de los prototipos creados por Figma, se anexan algunos ejemplos resultados de Figma.

- Nota: Esta herramienta solo fue usada para generar una retroalimentacion en el apartado del diseno pero se mantuvieron las ideas originales creadas por el equipo.



CATÁLOGO DE SERVICIOS

# Nuestros servicios

8 servicios especializados en 3 categorías

Todos

Redes (3)

Ciberseguridad (3)

Desarrollo de Software (2)



Redes

## Diseño de Infraestructura de Red

Planificación, implementación y documentación de redes LAN/WAN/SD-WAN empresariales.

🕒 2-6 semanas

★ 4.9



Redes

## Monitoreo de Red 24/7

Supervisión continua de la infraestructura de red con alertas en tiempo real.

🕒 Servicio mensual

★ 4.8



Ciberseguridad

## Auditoría de Seguridad (Pentest)

Pruebas de penetración y evaluación exhaustiva de vulnerabilidades en tu infraestructura.

🕒 1-3 semanas

★ 5



Ciberseguridad

## Implementación SIEM

Despliegue y configuración de plataformas de gestión de eventos de seguridad.

🕒 3-8 semanas

★ 4.7



Ciberseguridad

## Gestión de Identidades (IAM)

Implementación de controles de acceso basados en roles y autenticación multifactor.

🕒 2-5 semanas

★ 4.8



Desarrollo de Software

## Desarrollo de Aplicaciones Web Seguras

Construcción de aplicaciones web con OWASP Top 10 integrado desde el diseño.

🕒 Según alcance

★ 4.9

PANEL DE CONTROL

Última actualización: 10 jun 2026, 09:42

Actualizar

# Estadísticas del portal



84

Total solicitudes  
+12% vs mes anterior



8

Servicios activos  
Catálogo completo



23

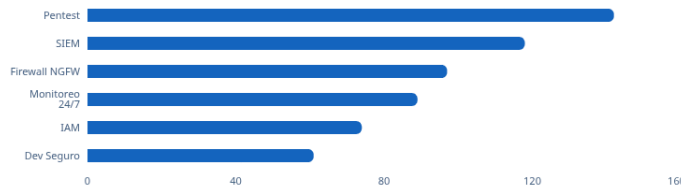
Cientes este mes  
+5 vs mayo



79%

Tasa de cierre  
+4% vs mes anterior

## Servicios más consultados



## Solicitudes por categoría



## 12. Paleta de colores y tipografía

| Color            | Codigo Hexadecimal | Elemento                                                                                                                   |
|------------------|--------------------|----------------------------------------------------------------------------------------------------------------------------|
| Azul marino      | #0C2340            | Barra de navegación (navbar) y fondo principal del banner/hero de inicio                                                   |
| Azul tecnologico | #1565C0            | Botones principales (Ver Servicios, Solicitar Cotización, botones Bootstrap), encabezados destacados y elementos de acción |
| Verde turquesa   | #00C2A0            | Tarjetas de servicios, tarjetas de misión/visión en la sección Nosotros, elementos destacados                              |
| Gris claro       | #F4F6F9            | Fondo general de la página (body)                                                                                          |
| Azul claro       | #E8F0FE            | Fondos secundarios, botones de filtros de categorías y elementos suaves de interfaz                                        |

La identidad visual del sistema SeguriTech utiliza una paleta basada en tonos azules y verdes asociados al sector tecnológico y de ciberseguridad. El azul representa confianza y seguridad, mientras que el verde representa innovación y crecimiento. Los colores secundarios permiten mantener una interfaz limpia, organizada y con buena jerarquía visual.

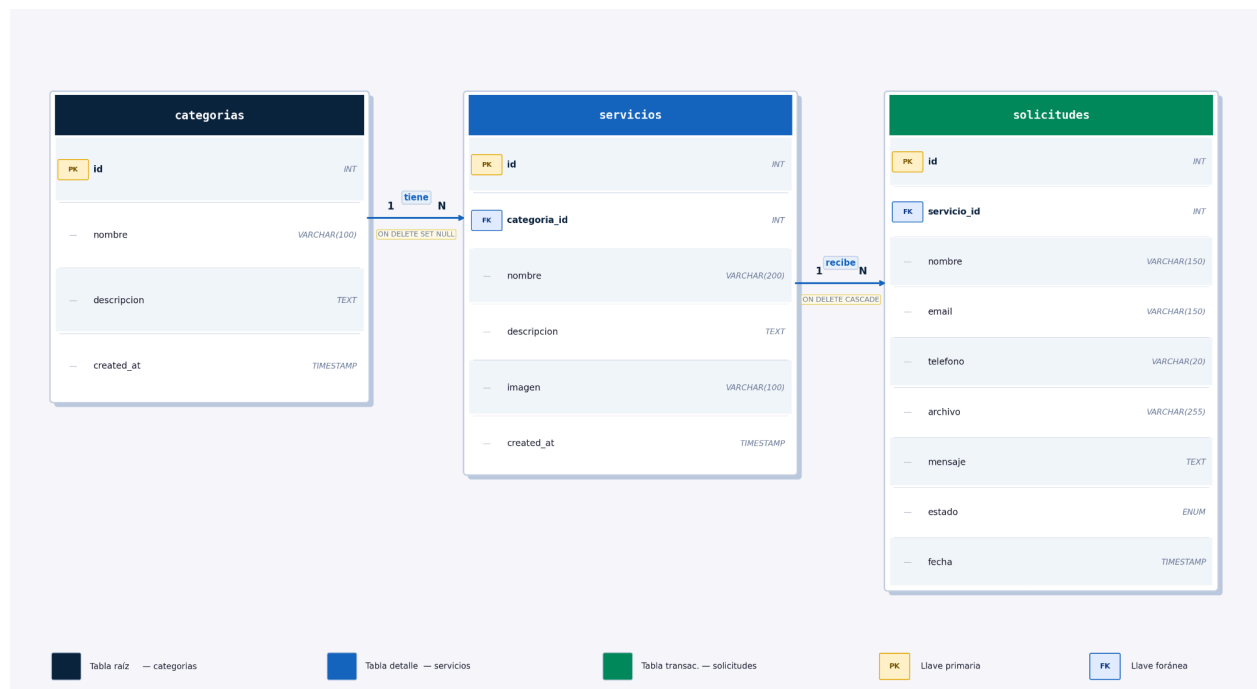
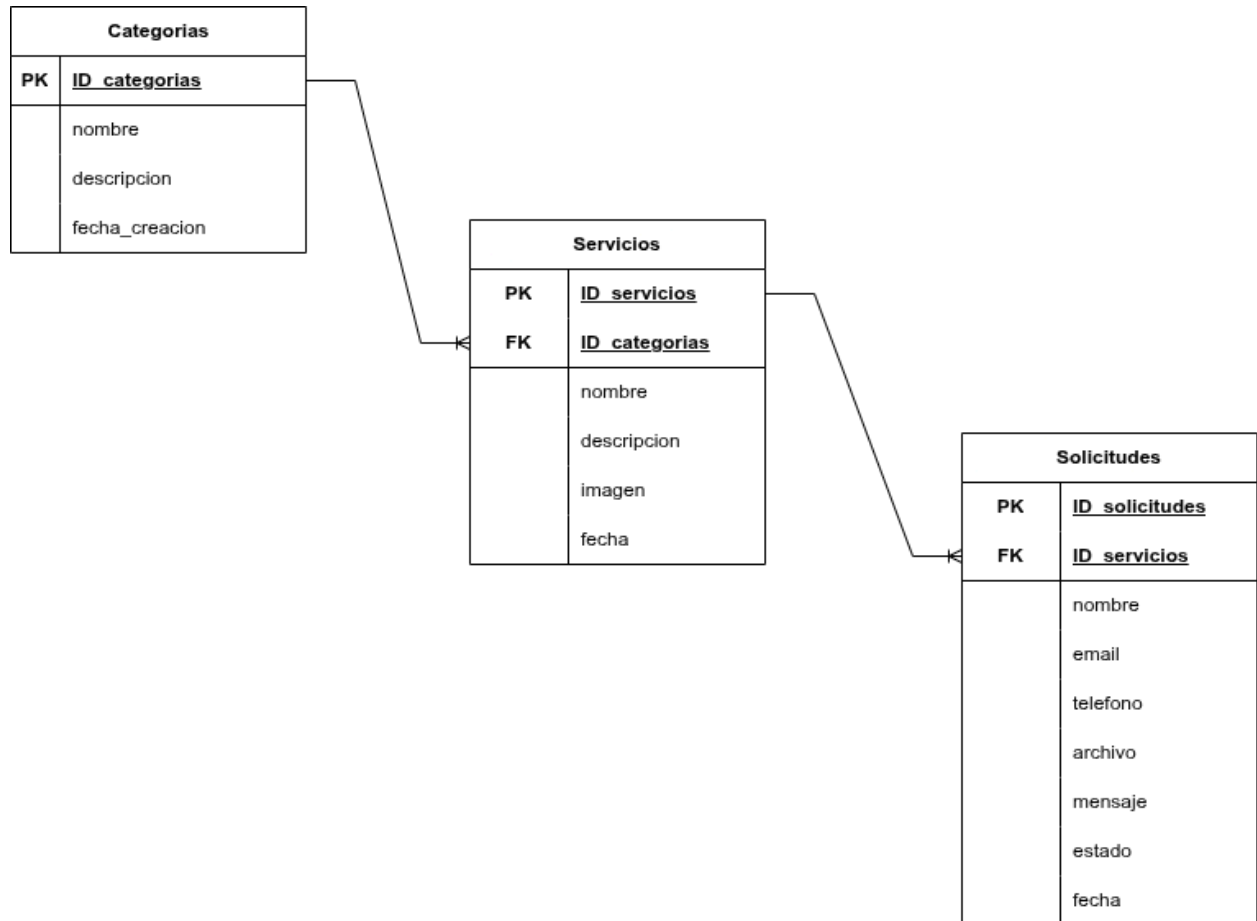
La interfaz utiliza la tipografía “Inter” como fuente principal, debido a que está diseñada para aplicaciones digitales y sistemas web. Su estructura permite una correcta legibilidad en diferentes tamaños de pantalla y facilita la jerarquización de contenidos mediante distintos pesos tipográficos.



| Elemento         | Tamano | CSS             |
|------------------|--------|-----------------|
| Texto normal     | 400    | body            |
| Botones y menú   | 500    | .btn, .nav-link |
| Títulos pequeños | 600    | h4, h5          |
| Secciones        | 700    | h2, h3          |
| Título principal | 800    | h1              |

## 13. Modelo relacional

Para la creación del diagrama entidad-relación se usó draw.io (también conocido como diagrams.net) que es una aplicación de diagramación gratuita y de código abierto que permite crear diversos tipos de diagramas, como flujos, UML, ER, redes, organigramas y mapas mentales. Desarrollada por JGraph, es una herramienta segura y flexible que funciona directamente en el navegador web o puede descargarse para uso offline en Windows, macOS y Linux.



| Campo               | Detalle       |
|---------------------|---------------|
| Nombre              | seguritech_db |
| Total de tablas     | 3             |
| Total de relaciones | 2             |

Tabla 1 — categorias

Almacena las categorías que agrupan los servicios ofrecidos por SeguriTech. Es la tabla raíz del modelo, ya que ninguna otra tabla la referencia excepto servicios.

| Campo       | Tipo         | Restricción               | Descripción                                        |
|-------------|--------------|---------------------------|----------------------------------------------------|
| id          | INT          | PK, AUTO_INCREMENT        | Identificador único de la categoría                |
| nombre      | VARCHAR(100) | NOT NULL, UNIQUE          | Nombre de la categoría (no se permiten duplicados) |
| descripcion | TEXT         | —                         | Descripción general de los servicios que agrupa    |
| created_at  | TIMESTAMP    | DEFAULT CURRENT_TIMESTAMP | Fecha y hora de registro automática                |

- Datos de prueba incluidos: 4 categorías — Redes de Cómputo, Ciberseguridad, Desarrollo de Software, Capacitación.

## Tabla 2 — servicios

Contiene el catálogo completo de servicios tecnológicos que SeguriTech ofrece a sus clientes. Depende de categorias mediante una llave foránea.

| Campo        | Tipo         | Restricción                  | Descripción                              |
|--------------|--------------|------------------------------|------------------------------------------|
| id           | INT          | PK, AUTO_INCREMENT           | Identificador único del servicio         |
| categoria_id | INT          | FK → categorias(id)          | Categoría a la que pertenece el servicio |
| nombre       | VARCHAR(200) | NOT NULL                     | Nombre descriptivo del servicio          |
| descripcion  | TEXT         | NOT NULL                     | Descripción detallada del servicio       |
| imagen       | VARCHAR(100) | NOT NULL                     | Nombre del archivo de imagen asociado    |
| created_at   | TIMESTAMP    | DEFAULT<br>CURRENT_TIMESTAMP | Fecha y hora de registro automática      |

- Regla de integridad: ON DELETE SET NULL — si se elimina una categoría, los servicios asociados no se eliminan; su categoria\_id se establece en NULL, preservando el historial del catálogo.
- Datos de prueba incluidos: 10 servicios distribuidos entre las 4 categorías.

### Tabla 3 — solicitudes

Es la tabla transaccional central del sistema. Registra cada solicitud de servicio enviada por un cliente a través del portal web. Consolida los datos del solicitante y del servicio en un solo registro para simplificar las operaciones Create/Read del sistema.

| Campo       | Tipo         | Restricción           | Descripción                         |
|-------------|--------------|-----------------------|-------------------------------------|
| id          | INT          | PK,<br>AUTO_INCREMENT | Identificador único de la solicitud |
| servicio_id | INT          | FK → servicios(id)    | Servicio que se está solicitando    |
| nombre      | VARCHAR(150) | NOT NULL              | Nombre completo del solicitante     |
| email       | VARCHAR(150) | NOT NULL              | Correo electrónico de contacto      |
| telefono    | VARCHAR(20)  | NOT NULL              | Número telefónico de contacto       |
| archivo     | VARCHAR(255) | —                     | Ruta del archivo adjunto (opcional) |

|         |           |                           |                                                                  |
|---------|-----------|---------------------------|------------------------------------------------------------------|
| mensaje | TEXT      | NOT NULL                  | Descripción de la necesidad del cliente                          |
| estado  | ENUM      | DEFAULT 'pendiente'       | Estado del proceso: pendiente, en_proceso, completada, cancelada |
| fecha   | TIMESTAMP | DEFAULT CURRENT_TIMESTAMP | Fecha y hora de envío automática                                 |

- Regla de integridad: ON DELETE CASCADE — si se elimina un servicio del catálogo, todas sus solicitudes asociadas se eliminan también, manteniendo la consistencia referencial.
- Datos de prueba incluidos: 15 solicitudes con distintos estados para demostrar las funcionalidades del dashboard.

La relación categorías → servicios es de uno a muchos: una categoría agrupa varios servicios, pero cada servicio pertenece a una sola categoría.

La relación servicios → solicitudes también es de uno a muchos: un servicio puede recibir múltiples solicitudes, pero cada solicitud referencia un único servicio.

## 14. Tecnologías empleadas

Para el desarrollo del sistema web SeguriTech: Catálogo de Servicios Tecnológicos y Ciberseguridad, se utilizaron tecnologías orientadas al desarrollo web moderno, siguiendo una arquitectura cliente-servidor basada en el patrón MVC (Modelo-Vista-Controlador). La selección tecnológica permitió implementar una aplicación dinámica, escalable y con separación de responsabilidades entre la interfaz, lógica de negocio y acceso a datos.

## 14.1. Frontend

- **HTML5:** Se utilizó HTML5 para la construcción de la estructura de las vistas del sistema mediante archivos **EJS**, aplicando etiquetas semánticas como:
  - <header> para la sección de navegación principal.
  - <nav> para el menú de navegación.
  - <section> para organizar los apartados principales.
  - <article> para componentes independientes como tarjetas de información.
  - <footer> para la información complementaria.

Esto permite mejorar la organización del contenido, accesibilidad y mantenimiento del código.

- **CSS3:** Se utilizó CSS3 para la personalización visual de la aplicación, definiendo:
  - Paleta de colores corporativa.
  - Tipografías.
  - Diseño de tarjetas.
  - Espaciados y distribución de elementos.
  - Adaptación responsiva para diferentes tamaños de pantalla.

Además, se implementaron variables CSS para mantener una estructura organizada del diseño:

- **Bootstrap 5:** Se utilizó Bootstrap como framework de diseño frontend para facilitar:
  - Diseño responsivo.
  - Componentes visuales como botones, tarjetas, formularios y navegación.
  - Sistema de columnas mediante Grid.
  - Validaciones visuales y estilos predefinidos.

Esto permitió reducir tiempos de desarrollo y mantener una interfaz adaptable.

- **JavaScript:** Se utilizó JavaScript del lado del cliente para agregar comportamiento dinámico a la aplicación, incluyendo:
  - Validación de formularios.

- Manipulación del DOM.
- Carga dinámica de información.
- Comunicación con el servidor mediante solicitudes AJAX.

## 14.2. Backend

- **Node.js:** Node.js fue utilizado como entorno de ejecución para desarrollar el servidor web de la aplicación. Sus funciones principales fueron:
  - Manejo de solicitudes HTTP.
  - Ejecución de la lógica del sistema.
  - Comunicación con la base de datos.
  - Procesamiento de formularios.
  - Gestión de archivos enviados por usuarios.
- **Express.js:** Se utilizó Express.js como framework backend para Node.js debido a su facilidad para construir aplicaciones web mediante rutas y controladores. Sus funciones dentro del proyecto fueron:
  - Definición de rutas del sistema.
  - Manejo de peticiones GET y POST.
  - Integración con las vistas EJS.
  - Organización del proyecto bajo arquitectura MVC.
- **EJS (Embedded JavaScript):** EJS fue utilizado como motor de plantillas para generar páginas HTML dinámicas desde el servidor. Permitió:
  - Insertar información proveniente de la base de datos.
  - Mostrar servicios dinámicamente.
  - Reutilizar componentes mediante archivos parciales.

## 14.3. Base de datos

**MySQL** fue utilizado como sistema gestor de base de datos relacional para almacenar la información del sistema. Se utilizó para administrar:



- Servicios disponibles.
- Categorías de servicios.
- Solicitudes realizadas por usuarios.
- Información necesaria para generar estadísticas.

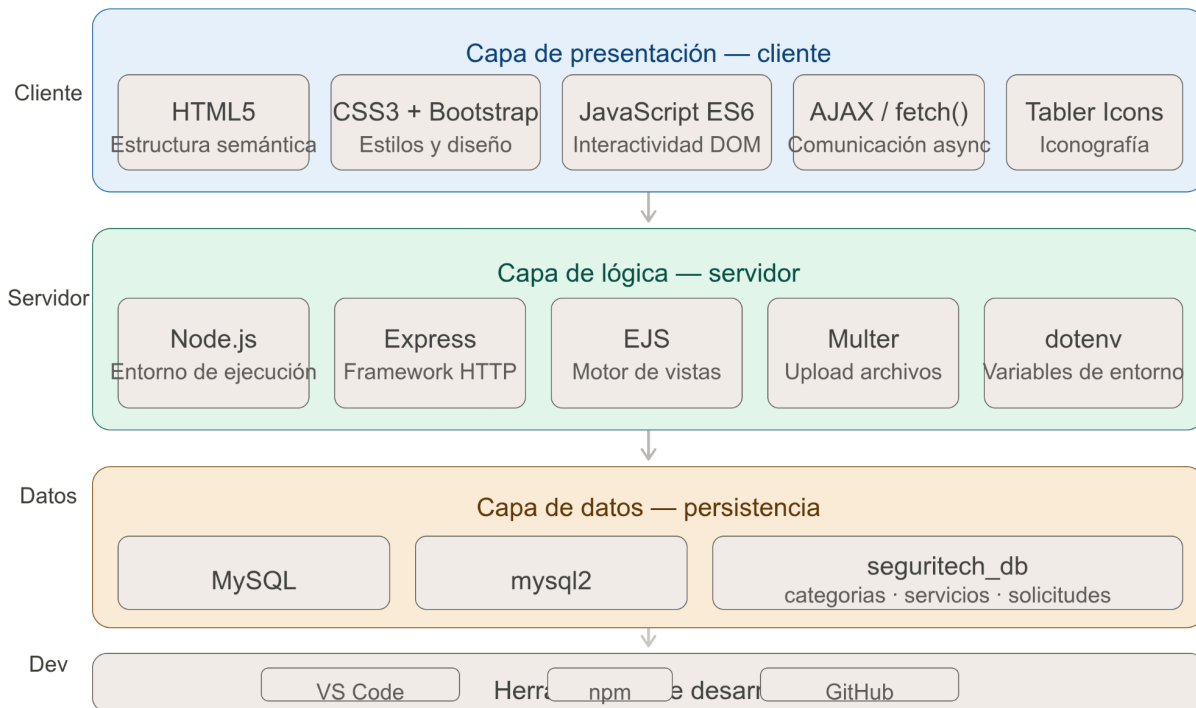
El modelo implementado sigue una estructura relacional donde las tablas mantienen relaciones mediante claves primarias y foráneas.

#### 14.4. Comunicación dinámica

- **AJAX / Fetch API:** Se implementó AJAX para permitir la comunicación entre cliente y servidor sin necesidad de recargar completamente las páginas. Fue utilizado para:
  - Obtener servicios desde la base de datos.
  - Actualizar información del dashboard.
  - Cargar estadísticas dinámicamente.

### 15. Diagrama del stack tecnológico

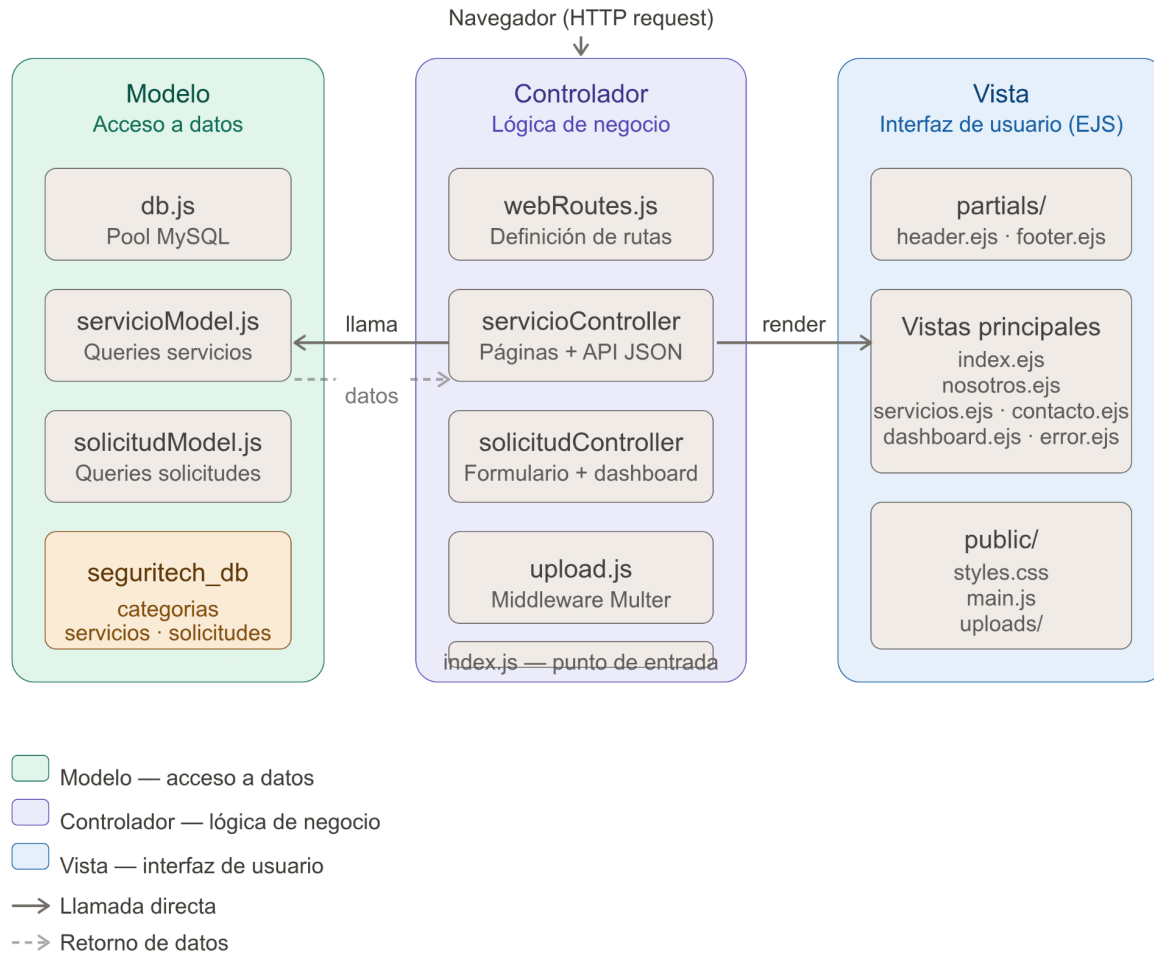
El stack tecnológico de SeguriTech está organizado en cuatro capas apiladas donde cada una depende de la inferior. La capa de cliente se encarga de lo que el usuario ve e interactúa: HTML5 para la estructura semántica, CSS3 con Bootstrap para los estilos responsivos, JavaScript para la interactividad del DOM, fetch() para la comunicación asíncrona. La capa de servidor procesa las peticiones con Node.js como entorno de ejecución, Express como framework HTTP, EJS como motor de plantillas, Multer para la gestión de archivos adjuntos y dotenv para las variables de entorno. La capa de datos persiste la información en MySQL, accedida desde Node.js mediante el driver mysql2 con soporte nativo para promesas y consultas parametrizadas. La capa de herramientas engloba el ecosistema de desarrollo: npm, Git/GitHub y VS Code.



## 16. Estructura MVC

El diagrama muestra cómo los archivos reales del proyecto se distribuyen entre las tres capas del patrón MVC. El flujo siempre parte del navegador, llega al Controlador a través de las rutas, el Controlador consulta al Modelo si necesita datos de la base de datos, y finalmente renderiza la Vista con esos datos para devolverla al cliente.

Los archivos del Modelo (`db.js`, `servicioModel.js`, `solicitudModel.js`) son los únicos que tienen contacto con la base de datos. Los archivos del Controlador (`webRoutes.js`, `servicioController.js`, `solicitudController.js`, `upload.js`) contienen toda la lógica de negocio y nunca generan HTML directamente. Los archivos de la Vista (`*.ejs` y `public/`) se limitan a presentar datos, sin consultar la base de datos ni contener lógica de negocio.



## 17. Explicación MVC

El Modelo representa los datos y las reglas de negocio. En SeguriTech, los modelos son `servicioModel.js` y `solicitudModel.js`. Cada uno contiene funciones que ejecutan consultas SQL parametrizadas contra la base de datos `seguritech_db` usando el pool de conexiones definido en `db.js`. Los modelos nunca reciben peticiones HTTP ni generan HTML; solo reciben parámetros, consultan la base de datos y devuelven resultados.

```
// Ejemplo: servicioModel.js — solo datos, sin lógica HTTP
getAllWithCategoria: async () => {
  const [rows] = await pool.query(
    `SELECT s.*, c.nombre AS categoria_nombre`
```

```

    FROM servicios s
    LEFT JOIN categorias c ON s.categoria_id = c.id`
  );
  return rows;
}

```

El Controlador es el intermediario entre el Modelo y la Vista. Recibe la petición HTTP desde las rutas definidas en `webRoutes.js`, solicita los datos necesarios al Modelo, aplica la lógica de negocio (validaciones, transformaciones) y decide qué Vista renderizar o qué JSON devolver. En SeguriTech hay dos controladores: `servicioController.js` gestiona las páginas de inicio, nosotros, servicios y contacto, además del endpoint `/api/servicios`; `solicitudController.js` gestiona el formulario de cotización, el dashboard y el endpoint `/api/dashboard/data`.

```

// Ejemplo: servicioController.js — orquesta modelo y vista
getServiciosPage: async (req, res, next) => {
  try {
    const servicios = await Servicio.getAllWithCategoria(); // llama al Modelo
    res.render('servicios', { titulo: 'Servicios', servicios }); // renderiza la Vista
  } catch (e) {
    next(e); // delega al manejador de errores
  }
}

```

La Vista presenta los datos al usuario sin contener lógica de negocio. En SeguriTech las vistas son archivos `.ejs` que reciben variables del controlador (servicios, titulo, etc.) y las renderizan usando la sintaxis `<%= %>` con escape automático para prevenir XSS. Los parciales `header.ejs` y `footer.ejs` se reutilizan en todas las páginas. Los archivos en `public/` (CSS y JS del cliente) son parte de la capa de Vista ya que solo afectan la presentación e interacción del usuario.

```

<!-- Ejemplo: servicios.ejs — solo presentación, sin SQL →
<% servicios.forEach(s => { %>
  <article class="card-servicio">
    <h5><%= s.nombre %></h5>
    <p><%= s.descripcion %></p>
  </article>
<% }) %>

```

Cuando un usuario abre /servicios en el navegador ocurre lo siguiente en orden: Express recibe la petición GET y la compara con las rutas en webRoutes.js, que la asigna a servicioController.getServiciosPage. El controlador llama a Servicio.getAllWithCategoria() en el modelo. El modelo ejecuta la consulta SQL con mysql2 y devuelve un array de objetos. El controlador llama a res.render('servicios', { servicios }). EJS compila servicios.ejs con los datos y genera el HTML final. Express envía el HTML al navegador.

Cuando el usuario filtra por categoría en la misma página ocurre una variante asíncrona: el JavaScript del cliente hace fetch('/api/servicios'), el controlador responde con res.json({ success: true, data: servicios }) en lugar de renderizar una vista, y el JS del cliente inserta las tarjetas en el DOM sin recargar la página.

## 18. Módulos implementados

### a) Módulo de Configuración del Servidor

Configuración inicial de Express, middlewares globales (CORS, parseo de JSON/urlencoded, archivos estáticos), motor de plantillas EJS y manejo de errores 404/500.

Archivos: index.js, .env

### b) Módulo de Conexión a Base de Datos

Pool de conexiones MySQL mediante mysql2/promise, configuración por variables de entorno.

Archivos: db.js, bd.sql

### c) Módulo de Rutas (Routing)

Definición de las rutas web (páginas) y rutas API (AJAX), separación entre vistas renderizadas en servidor y endpoints JSON.

Archivos: webRoutes.js

### d) Módulo de Controladores

Lógica de negocio: obtención de servicios para el catálogo, procesamiento de solicitudes de cotización, generación de datos del dashboard.

Archivos: servicioController.js, solicitudController.js

e) Módulo de Modelos (Acceso a Datos)

Consultas parametrizadas a MySQL para servicios, categorías y solicitudes (CRUD, conteos, agregaciones para el dashboard).

Archivos: servicioModel.js, solicitudModel.js

f) Módulo de Validación y Sanitización

Validación de campos del formulario de contacto (nombre, email, teléfono, mensaje, servicio existente) y sanitización contra XSS con la librería validator.

Archivo: solicitudController.js (lógica de validación)

g) Módulo de Carga de Archivos

Configuración de Multer para subir archivos adjuntos en las solicitudes: validación de extensión/MIME, límite de tamaño (5MB), manejo de errores de subida.

Archivo: upload.js

h) Módulo de Vistas (Frontend - EJS)

Plantillas del catálogo, página de inicio, "Nosotros", contacto y dashboard, con partials reutilizables (header/footer).

Archivos: views/\*.ejs, partials/

i) Módulo de Interactividad (Frontend - JavaScript)

Validación de formulario en cliente, envío AJAX de solicitudes, carga dinámica del catálogo de servicios con filtros por categoría, y dashboard con estadísticas en tiempo real vía fetch.

Archivo: main.js

j) Módulo de Estilos (CSS)

Variables de marca, estilos del hero, tarjetas de servicio, navbar y diseño responsive.

Archivo: styles2.css

## 19. Mecanismos de seguridad

### 19.1. Prevención de Inyección SQL (Consultas Parametrizadas)

Todas las consultas a la base de datos utilizan placeholders (?) en lugar de concatenar directamente los valores recibidos del usuario. La librería mysql2 se encarga de escapar

correctamente cada parámetro antes de enviarlo al motor de base de datos, evitando que un atacante pueda inyectar instrucciones SQL maliciosas a través de los formularios.

```
// solicitudModel.js
const [result] = await db.query(
  'INSERT INTO solicitudes (nombre, email, telefono, servicio_id, archivo, mensaje) VALUES'
  '(?, ?, ?, ?, ?, ?)',
  [data.nombre, data.email, data.telefono, data.servicio_id, data.archivo || null, data.mensaje]
);
```

*Figura 1. Inserción de una solicitud mediante consulta parametrizada.*

Justificación: separar los datos de la estructura de la consulta es la defensa estándar recomendada por OWASP contra ataques de inyección SQL (Top 10 – A03:2021 Injection).

## 19.2. Prevención de Cross-Site Scripting (XSS)

Se aplican dos capas de protección complementarias:

- Escape automático de EJS: al renderizar variables con `<%= %>`, EJS convierte automáticamente caracteres especiales (`<`, `>`, `&`, `"`) en sus entidades HTML, evitando que un usuario pueda inyectar `<script>` en las vistas.
- Sanitización en el backend con `validator.escape()`: los campos de texto libre (nombre, mensaje) se sanitizan antes de guardarse en la base de datos.

```
// solicitudController.js
const nombre = validator.escape(validator.trim(req.body.nombre || ""));
const mensaje = validator.escape(validator.trim(req.body.mensaje || ""));
```

*Figura 2. Sanitización de campos de texto libre antes de almacenarlos.*

Justificación: la combinación de sanitización en el servidor y escape automático en el motor de plantillas implementa el principio de defensa en profundidad frente a XSS (OWASP A03:2021).

## 19.3. Validación de Datos de Entrada

El servidor valida formato, longitud y existencia de cada campo antes de procesar la solicitud, independientemente de la validación realizada en el cliente, ya que el código JavaScript del navegador puede ser desactivado o evadido por el usuario.

```
// solicitudController.js
if (nombre.length < 3 || nombre.length > 150) {
  errores.push('Nombre debe tener entre 3 y 150 caracteres.');
```

```
}
if (!validator.isEmail(email)) {
  errores.push('Email inválido.');
```

```
}
if (!telefono || !/^[d\s\(-)+]{7,20}$/.test(telefono)) {
  errores.push('Teléfono inválido (mínimo 7 dígitos).');
```

```
}
if (!servicio_id || !validator.isInt(servicio_id, { min: 1 })) {
  errores.push('Selecciona un servicio válido.');
```

```
} else {
  const servicioValido = await Servicio.findById(parseInt(servicio_id));
  if (!servicioValido) errores.push('El servicio seleccionado no existe.');
```

```
}
```

*Figura 3. Validación de campos del formulario de cotización en el servidor.*

Justificación: la validación del lado del servidor es el único punto de control que un atacante no puede evadir. Verificar que `servicio_id` exista realmente en la base de datos evita además referencias inválidas o manipulación del formulario con valores arbitrarios.

## 19.4. Validación y Restricción de Archivos Subidos

El middleware de Multer restringe los archivos que pueden subirse mediante tres filtros combinados: extensión permitida, tipo MIME permitido y tamaño máximo. Además, los archivos se renombran con un identificador aleatorio para evitar colisiones y ocultar el nombre original.

```
// upload.js
const allowedExts = ['.jpg', '.jpeg', '.png', '.pdf', '.doc', '.docx'];
const allowedMimes = ['image/jpeg', 'image/png', 'application/pdf',
  'application/msword',
  'application/vnd.openxmlformats-officedocument.wordprocessingml.document'];

const ext = path.extname(file.originalname).toLowerCase();
if (allowedExts.includes(ext) && allowedMimes.includes(file.mimetype)) cb(null, true);
else cb(new Error('Solo JPG, PNG, PDF, DOC, DOCX'));
```

```
const upload = multer({
  storage, fileFilter,
  limits: { fileSize: parseInt(process.env.MAX_FILE_SIZE) || 5 * 1024 * 1024, files: 1 }
});
```



*Figura 4. Filtro de extensión, tipo MIME y límite de tamaño para archivos adjuntos.*

```
// upload.js - generación de nombre aleatorio
filename: (req, file, cb) => {
  const unique = Date.now() + '-' + Math.round(Math.random() * 1E9);
  cb(null, 'seguritech-' + unique + path.extname(file.originalname).toLowerCase());
}
```

*Figura 5. Renombrado aleatorio de archivos para evitar colisiones y exposición de nombres originales.*

Justificación: validar extensión y tipo MIME (no solo uno de los dos) reduce el riesgo de subir archivos ejecutables disfrazados (por ejemplo, un .php renombrado a .jpg). El límite de tamaño previene ataques de denegación de servicio por agotamiento de espacio en disco, y el renombrado evita ataques de path traversal y colisiones de nombres.

## 19.5. Gestión de Variables de Entorno (Credenciales)

Las credenciales de la base de datos y otros valores de configuración sensibles no se escriben directamente en el código fuente, sino que se cargan desde un archivo .env mediante la librería dotenv.

```
// db.js
const pool = mysql.createPool({
  host: process.env.DB_HOST || 'localhost',
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASS || '',
  database: process.env.DB_NAME || 'seguritech_db',
  port: process.env.DB_PORT || 3306,
  ...
});
```

*Figura 6. Configuración del pool de conexiones a partir de variables de entorno.*

Justificación: mantener las credenciales fuera del código fuente evita que se expongan si el repositorio se publica en un servicio como GitHub, y permite usar configuraciones distintas entre entornos de desarrollo y producción sin modificar el código.

## 19.6. Manejo Centralizado de Errores

La aplicación captura errores no controlados mediante un middleware de errores de Express, mostrando al usuario una página genérica en lugar de exponer detalles técnicos (rutas del servidor, mensajes de excepción, consultas SQL) que podrían ayudar a un atacante.

```
// index.js
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).render('error', { titulo: 'Error', mensaje: 'Ocurrió un error.' });
});
```

*Figura 7. Middleware de manejo centralizado de errores.*

Justificación: este mecanismo evita la divulgación de información sensible del sistema (OWASP A05:2021 Security Misconfiguration), mientras se conserva el registro del error en consola para depuración por parte del desarrollador.

## 19.7. Política de Acceso CORS

La aplicación define explícitamente la política de Cross-Origin Resource Sharing (CORS) mediante el middleware cors, controlando qué orígenes pueden realizar peticiones a la API.

```
// index.js
app.use(cors());
```

*Figura 8. Configuración del middleware CORS.*

Justificación: declarar explícitamente la política CORS es el punto de control necesario para, en un entorno de producción, restringir el acceso de la API solo a los dominios autorizados, en lugar de depender del comportamiento por defecto del navegador.

## 19.8. Integridad Referencial en la Base de Datos

El esquema de base de datos utiliza llaves foráneas con políticas ON DELETE definidas, garantizando consistencia de datos a nivel de motor de base de datos y no solo a nivel de aplicación.

```
-- bd.sql
FOREIGN KEY (categoria_id) REFERENCES categorias(id) ON DELETE SET NULL
...
FOREIGN KEY (servicio_id) REFERENCES servicios(id) ON DELETE CASCADE
```

Figura 9. Restricciones de llave foránea en el esquema de la base de datos.

Justificación: estas restricciones evitan registros huérfanos, por ejemplo solicitudes que apunten a servicios inexistentes, incluso si una capa superior de la aplicación presentara un error, añadiendo una capa adicional de integridad a nivel de datos.

## 19.9. Áreas de Mejora Identificadas

Como parte del análisis de seguridad del proyecto, se identificaron los siguientes aspectos que podrían fortalecerse en una siguiente iteración:

- Incorporar cabeceras de seguridad HTTP (CSP, X-Frame-Options, HSTS) mediante un middleware como Helmet.
- Habilitar HTTPS para cifrar el tráfico entre el cliente y el servidor.
- Implementar limitación de tasa (rate limiting) en el endpoint /solicitud para prevenir envíos masivos automatizados.
- Incorporar un token CSRF en el formulario de contacto.

## 20. Modelado de amenaza

| Categoría STRIDE        | Amenaza Específica Identificada                                                    | Componente / Ruta Afectada                  | Impacto en el Sistema                                                 | Mecanismo de Mitigación (Implementado o Propuesto)                                                         |
|-------------------------|------------------------------------------------------------------------------------|---------------------------------------------|-----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| Spoofing (Suplantación) | Un atacante intercepta la comunicación o envía solicitudes falsas haciéndose pasar | Formulario de Cotización / Peticiones HTTP. | Captación de datos falsos en la base de datos y pérdida de confianza. | Propuesto: Implementar HTTPS en producción y añadir control de sesiones seguras ( <b>express-session</b> ) |

|                           |                                                                                                                                                                      |                                                                                                                                              |                                                                                                                  |                                                                                                                                                                                   |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                           | por un cliente legítimo.                                                                                                                                             |                                                                                                                                              |                                                                                                                  | con cookies cifradas para la administración.                                                                                                                                      |
| Tampering<br>(Alteración) | Inyección SQL: Manipulación de entradas en formularios o en el filtrado dinámico para alterar o borrar datos de la BD.                                               | Modelos de datos ( <code>servicioMod</code><br><code>el.js</code> , <code>solicitudMod</code><br><code>el.js</code> ) y endpoints de la API. | Pérdida, alteración o borrado total de la información de la base de datos <code>seguritech_db</code> .           | Implementado: Uso estricto de consultas parametrizadas con placeholders (?) mediante el driver <code>mysql2</code> .                                                              |
| Tampering<br>(Alteración) | Cross-Site Scripting (XSS): Inyección de scripts maliciosos ( <code>&lt;script&gt;</code> ) a través de los campos de texto para alterar la vista de otros usuarios. | Vistas dinámicas del sistema generadas con el motor de plantillas.                                                                           | Ejecución de código arbitrario en el navegador de los usuarios o del administrador, comprometiendo la sesión.    | Implementado: Uso de sintaxis de escape automático de EJS ( <code>&lt;%= %&gt;</code> ) que neutraliza cualquier etiqueta HTML o script antes de renderizar.                      |
| Repudiation<br>(Repudio)  | Un usuario malintencionado realiza un envío masivo de cotizaciones falsas y niega haberlo hecho al no existir registros de auditoría.                                | Controlador de solicitudes ( <code>solicitudCo</code><br><code>ntroller.js</code> ).                                                         | Imposibilidad de rastrear el origen de un incidente o bloquear direcciones IP atacantes por falta de evidencias. | Propuesto: Integrar un middleware de registro de eventos (como <code>morgan</code> o <code>winston</code> ) para almacenar logs de peticiones con IP y timestamps en el servidor. |

|                                               |                                                                                                                                            |                                                                                          |                                                                                                         |                                                                                                                                                                                                                                                      |
|-----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Information Disclosure<br><br>(Fuga de datos) | Fuga por Errores:<br>Exposición de variables confidenciales o de la estructura interna de la BD a través de mensajes de error en pantalla. | Rutas del backend y middleware de manejo de errores.                                     | Revelación de credenciales a atacantes o detalles de la arquitectura que faciliten un hackeo posterior. | Implementado/Propuesto: Uso de archivos <code>.env</code> mediante <code>dotenv</code> para aislar credenciales. Se complementa con el middleware de errores centralizado y la vista <code>error.ejs</code> para ocultar fallos internos al cliente. |
| Denial of Service<br><br>(Denegación)         | Agotamiento de Almacenamiento : Un atacante sube múltiples archivos de gran tamaño repetidamente para llenar el disco del servidor.        | Módulo de subida de archivos (Formulario de Cotización).                                 | Caída del servidor web y bloqueo general del sistema por falta de espacio en disco.                     | Implementado: Configuración del middleware Multer para restringir estrictamente el tamaño máximo de los archivos a 5MB por subida.                                                                                                                   |
| Denial of Service<br><br>(Denegación)         | Inundación de Peticiones (HTTP Flood): Automatización de miles de consultas HTTP asíncronas para saturar el servidor.                      | Endpoints asíncronos ( <code>/api/servicios</code> y <code>/api/dashboard/data</code> ). | Inaccesibilidad del portal web para los clientes reales de SeguriTech (CTOs, PyMEs, etc.).              | Propuesto: Implementar un limitador de tasa de peticiones (Rate Limiting) con la librería <code>express-rate-limit</code> en las rutas críticas del backend.                                                                                         |
| Elevation of Privilege<br><br>(Elevación)     | Un visitante casual o atacante accede directamente a la URL del panel                                                                      | Ruta del Panel Estadístico / Dashboard administrativo                                    | Acceso no autorizado a métricas comerciales sensibles, volúmenes de                                     | Propuesto: (Identificado en el trabajo futuro del proyecto) Crear un sistema de autenticación                                                                                                                                                        |

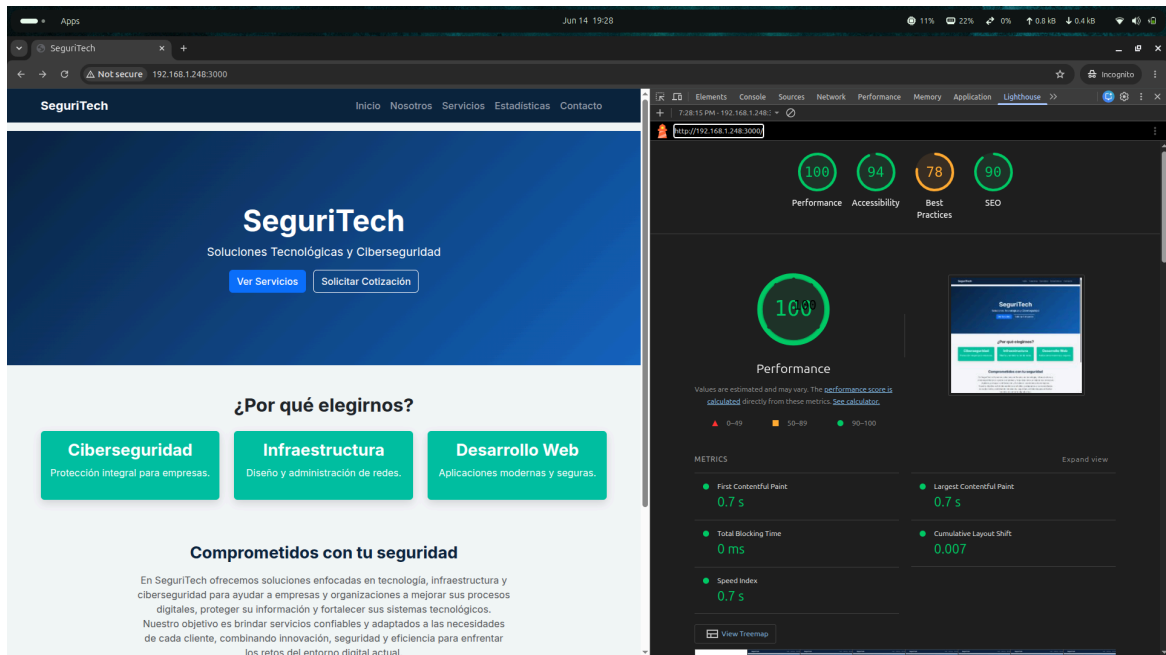
|  |                                |                 |                                  |                                                                                    |
|--|--------------------------------|-----------------|----------------------------------|------------------------------------------------------------------------------------|
|  | estadístico sin restricciones. | o (/dashboard). | solicitudes y datos de clientes. | basado en Roles de Usuario para restringir el acceso a las vistas administrativas. |
|--|--------------------------------|-----------------|----------------------------------|------------------------------------------------------------------------------------|

El proyecto que realizamos cuenta con un excelente nivel de seguridad inicial (nivel de desarrollo local localhost:3000) gracias a la separación de responsabilidades que otorga la arquitectura MVC, el uso de consultas preparadas y el escape de datos en plantillas. Para dar el salto a un entorno de producción seguro, las prioridades críticas de mitigación deben ser la implementación de autenticación/roles para proteger el Dashboard y la habilitación de HTTPS para el cifrado del tráfico de red.

## 21. Evidencias de pruebas

### 21.1. Auditoría con Google Lighthouse

La auditoría evalúa cuatro categorías: Rendimiento (Performance), Accesibilidad (Accessibility), Buenas Prácticas (Best Practices) y SEO. La siguiente captura muestra los resultados obtenidos después de aplicar las optimizaciones descritas en la sección 2:



## 21.2. Resultados por Categoría

- Rendimiento (Performance): 100/100. Esta es la categoría donde más impacto tuvieron las correcciones aplicadas. Antes de las optimizaciones, la ruta crítica de carga superaba los 2.4 segundos debido a recursos bloqueantes (Bootstrap, fuentes y main.js sin defer); después de aplicar defer, preconnect y compresión, la página obtiene el puntaje máximo.
- Accesibilidad (Accessibility): 94/100. Puntaje alto, aunque indica que existen detalles menores pendientes, probablemente relacionados con contraste de color en textos secundarios (por ejemplo, clases text-muted) o etiquetas faltantes en algunos elementos interactivos.
- Buenas Prácticas (Best Practices): 78/100. Es la categoría con menor puntaje. La causa principal observable es que el sitio se sirve por HTTP sin cifrado (indicador “Not secure” en la barra del navegador), lo cual Lighthouse penaliza de forma importante en esta categoría.
- SEO: 90/100. Puntaje bueno; suelen faltar pequeños detalles como una metaetiqueta de descripción (meta description) o textos de enlace más descriptivos para los motores de búsqueda.

## 21.3. Interpretación de las Métricas de Rendimiento

El panel de métricas (Core Web Vitals) reporta los siguientes valores:

- First Contentful Paint (FCP): 0.7 s — tiempo hasta que se pinta el primer elemento visible. Un valor por debajo de 1.8 s se considera bueno.
- Largest Contentful Paint (LCP): 0.7 s — tiempo hasta que se renderiza el elemento más grande visible (en este caso, la sección hero). Valores por debajo de 2.5 s se consideran buenos.
- Total Blocking Time (TBT): 0 ms — indica que ningún script bloquea la interacción del usuario durante la carga, resultado directo de haber agregado el atributo defer a los scripts.
- Cumulative Layout Shift (CLS): 0.007 — prácticamente nulo; corresponde al pequeño salto visual al cargar la fuente “Inter” documentado previamente, pero está muy por debajo del umbral de 0.1 que Lighthouse considera aceptable.
- Speed Index: 0.7 s — mide la rapidez con la que se muestra visualmente el contenido de la página durante la carga.

## 21.4. Prueba de solicitud

Se puso a prueba el correcto funcionamiento del apartado de “solicitud”. Simulando una cotización dentro del proyecto web.



## Solicitar Cotización

**Nombre completo \***

Abraham Nunez Prueba

**Email \***

correo@gmail.com

**Teléfono \***

123456789

**Servicio \***

[Desarrollo de Software] Desarrollo de Software de Cibe ▾

**Archivo adjunto**

Seleccionar archivo

Sartre - El existe...n humanismo.pdf

Formatos permitidos: JPG, PNG, PDF, DOC, DOCX. Máximo 5 MB.

**Mensaje \***

Hola, necesito ayuda urgentemente.



Enviar Solicitud

Aca solo se probó siguiendo la forma correcta de mandar el formulario (respetando parametros). Se verifica que se envíe exitosamente y que nos devuelve un número de folio.

## Solicitar Cotización

Solicitud enviada correctamente. Folio #33

Nombre completo \*

Email \*

Se confirma el envío de la solicitud llendo al apartado de “Estadísticas” y viendo la propia base de datos.

### Últimas 5 Solicitudes

| ID  | Cliente                             | Servicio                                 | Estado    | Fecha       |
|-----|-------------------------------------|------------------------------------------|-----------|-------------|
| #33 | Abraham Nunez Prueba                | Desarrollo de Software de Ciberseguridad | Pendiente | 14 jun 2026 |
| #32 | Skull                               | Pentesting                               | Pendiente | 14 jun 2026 |
| #31 | &#x27;; DROP TABLE servicios; --    | Pentesting                               | Pendiente | 14 jun 2026 |
| #30 | Juan&#x27; OR &#x27;1&#x27;=&#x27;1 | Pentesting                               | Pendiente | 14 jun 2026 |
| #29 | Skull                               | Desarrollo de Software de Ciberseguridad | Pendiente | 14 jun 2026 |

```
MariaDB [seguritech_db]> select * from solicitudes where nombre="Abraham Nunez Prueba";
+-----+-----+-----+-----+-----+-----+-----+-----+
| id | nombre | email | telefono | servicio_id | archivo | mensaje | estado | fecha |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 33 | Abraham Nunez Prueba | correo@gmail.com | 123456789 | 5 | seguritech-1781497727317-46749071.pdf | Hola, necesito ayuda urgentemente. | pendiente | 2026-06-14 22:28:47 |
+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.009 sec)
```

## 21.5. Prueba de solicitud erronea

Ahora se probara que el envio de formulario pida ciertos parametros al usuario. Principalmente en la seccion “nombre”, “correo”, “servicio”, “tamaño del archivo de evidencia” y “descripción”.

Si se intenta enviar un formulario vacio se muestra lo siguiente:

### Solicitar Cotización

**Nombre completo \***

!

Mínimo 3 caracteres.

**Email \***

!

Email inválido.

**Teléfono \***

!

Mínimo 7 dígitos.

**Servicio \***

Selecciona un servicio

!

▼

Selecciona un servicio.

**Archivo adjunto**

Seleccionar archivo

Sin archivos seleccionados

Formatos permitidos: JPG, PNG, PDF, DOC, DOCX. Máximo 5 MB.

**Mensaje \***

!

Mínimo 10 caracteres.

Enviar Solicitud

Si se intenta enviar un archivo de evidencia que supere los 5 MB, se muestra lo siguiente:

## Solicitar Cotización

Error al enviar la solicitud.

**Nombre completo \***

**Email \***

**Teléfono \***

**Servicio \***  

[Capacitación] Capacitación de Ciberseguridad ▾

**Archivo adjunto**  

Seleccionar archivo

Security+® Practice Tests.pdf

Formatos permitidos: JPG, PNG, PDF, DOC, DOCX. Máximo 5 MB.

**Mensaje \***  

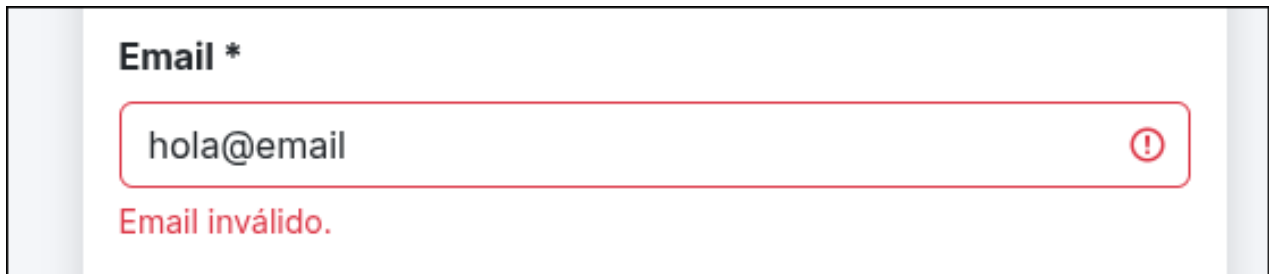
Ayudaaaaaaaaaaaaaa



Enviar Solicitud

## 21.6. Email falso

No se permite enviar un formulario con un email con el formato erroneo.

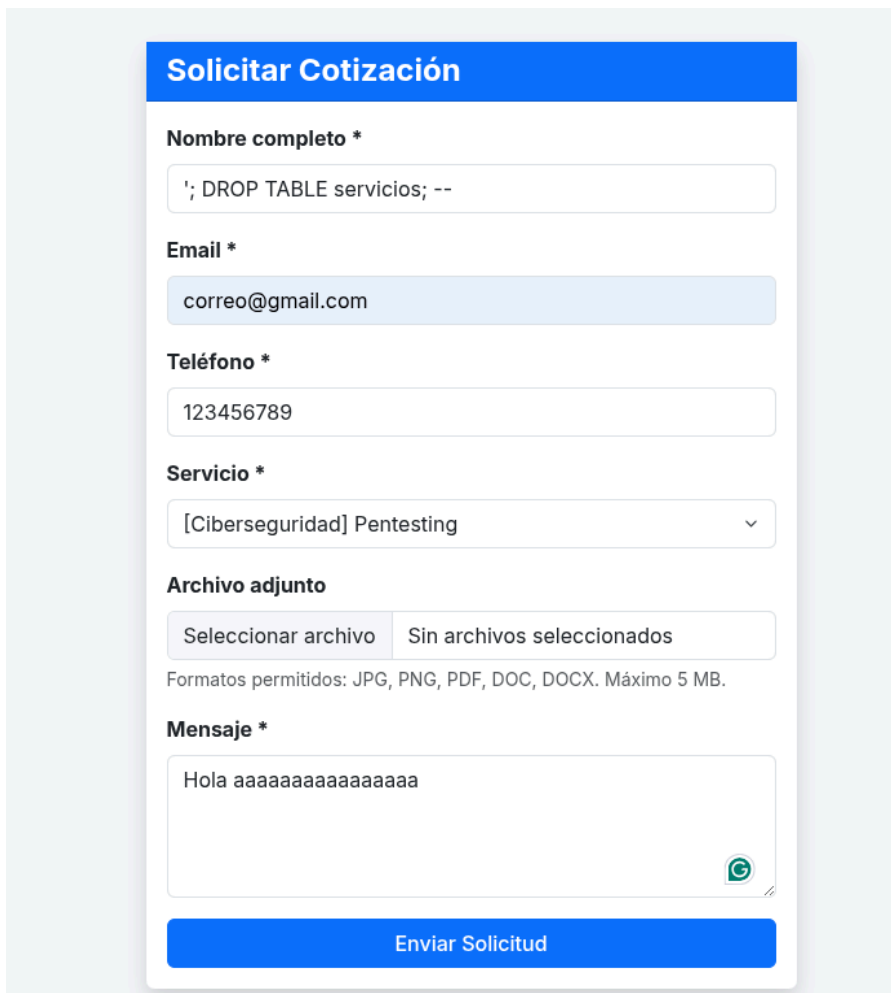


A screenshot of a web form section titled "Email \*". Below the title is a text input field containing the text "hola@email". To the right of the input field is a red circular icon with a white exclamation mark. Below the input field, the text "Email inválido." is displayed in red.

## 21.7. Pruebas de Inyección de SQL y Cross-Site Scripting (XSS)

En el formulario se pusieron comandos para verificar la integridad del proyecto web.

- Inyección de SQL



A screenshot of a web form titled "Solicitar Cotización". The form contains several fields: "Nombre completo \*" with the value "'; DROP TABLE servicios; --", "Email \*" with the value "correo@gmail.com", "Teléfono \*" with the value "123456789", "Servicio \*" with a dropdown menu showing "[Ciberseguridad] Pentesting", "Archivo adjunto" with a button "Seleccionar archivo" and the text "Sin archivos seleccionados", and "Mensaje \*" with the value "Hola aaaaaaaaaaaaaaa". Below the "Archivo adjunto" section, there is a note: "Formatos permitidos: JPG, PNG, PDF, DOC, DOCX. Máximo 5 MB." At the bottom of the form is a blue button labeled "Enviar Solicitud".

| Últimas 5 Solicitudes |                                     |                                          |           |             |
|-----------------------|-------------------------------------|------------------------------------------|-----------|-------------|
| ID                    | Cliente                             | Servicio                                 | Estado    | Fecha       |
| #31                   | &#x27;; DROP TABLE servicios; --    | Pentesting                               | Pendiente | 14 jun 2026 |
| #30                   | Juan&#x27; OR &#x27;1&#x27;=&#x27;1 | Pentesting                               | Pendiente | 14 jun 2026 |
| #29                   | Skull                               | Desarrollo de Software de Ciberseguridad | Pendiente | 14 jun 2026 |
| #28                   | Skull                               | Desarrollo de Software de Ciberseguridad | Pendiente | 14 jun 2026 |
| #27                   | Juan&#x27; OR &#x27;1&#x27;=&#x27;1 | Desarrollo de Software de Ciberseguridad | Pendiente | 14 jun 2026 |

SeguriTech
Inicio
Nosotros
Servicios
Estadísticas
Contacto

## Catálogo de Servicios

Conoce nuestras soluciones tecnológicas.

Todos
Redes de Cómputo
Ciberseguridad
Desarrollo de Software
Capacitación

Redes de Cómputo

Implementación de Redes de Cómputo

Diseño, instalación y configuración de infraestructura de redes cableadas e inalámbricas.

Solicitar

Redes de Cómputo

Mantenimiento de Redes de Cómputo

Mantenimiento preventivo y correctivo para garantizar el óptimo funcionamiento de su red.

Solicitar

Ciberseguridad

Implementación de Seguridad en Redes

Protección integral con firewalls, IDS/IPS y segmentación de red.

Solicitar

Redes de Cómputo

Automatización de Redes

Automatización de tareas mediante scripts y herramientas como Ansible.

Solicitar

Desarrollo de Software

Desarrollo de Software de Ciberseguridad

Creación de herramientas personalizadas: scanners, dashboards, etc.

Solicitar

Ciberseguridad

Auditorías de Ciberseguridad

Evaluación de postura de seguridad: políticas, controles, cumplimiento normativo.

Solicitar

El formulario se envió correctamente pero el proyecto mitiga el intento de inyección de SQL, ya que no afecta la base de datos del proyecto. En este ejemplo se puso una consulta que debería borrar la tabla de “Servicios” en la base de datos pero no surte efecto.

- Cross-Site Scripting (XSS)

Mensaje \*

<script>alert("XSS")</script>



Enviar Solicitud

```

32 | Skull | correo@gmail.com | 123456789 | 7 | NULL | &lt;script&gt;alert(&quot;XSS&quot;)&
| &#x2F;script&gt; | pendiente | 2026-06-14 21:23:33 |
33 | Abraham Nunez Prueba | correo@gmail.com | 123456789 | 5 | seguritech-1781497727317-46749071.pdf | Hola, necesito ayuda urgentemente.
| pendiente | 2026-06-14 22:28:47 |
+-----+-----+-----+-----+-----+-----+-----+
33 rows in set (0.001 sec)

MariaDB [seguritech db]>

```

Se puede observar que se guarda dentro de la base de datos sin dar ningun error dentro del proyecto.



Inicio

Nosotros

Servicios

Estadísticas

Contacto

Ver Servicios

Solicitar Cotización

## ¿Por qué elegirnos?

### Ciberseguridad

Protección integral para empresas.

### Infraestructura

Diseño y administración de redes



## Catálogo de Servicios

Conoce nuestras soluciones tecnológicas.

Todos

Redes de Cómputo

Ciberseguridad

Desarrollo de Software

Capacitación

Redes de Cómputo

### Implementación de Redes de Cómputo

Diseño, instalación y configuración de infraestructura de redes cableadas e inalámbricas.

## 22. Problemas encontrados y soluciones

Durante el desarrollo y la auditoría del proyecto se identificaron los siguientes problemas, junto con la solución implementada o propuesta para cada uno.

### 22.1. Duplicación completa del código en main.js

Durante el desarrollo y la auditoría del proyecto se identificaron los siguientes problemas, junto con la solución implementada o propuesta para cada uno.

Descripción del problema: durante la revisión del frontend se detectó que el archivo `public/js/main.js` contenía todo su contenido duplicado (las mismas funciones y listeners declarados dos veces). Esto provocaba tres fallos funcionales:

- El formulario de cotización registraba dos listeners de submit, por lo que cada envío generaba dos peticiones POST a `/solicitud`, pudiendo insertar solicitudes duplicadas en la base de datos.
- El dashboard ejecutaba dos peticiones simultáneas a `/api/dashboard/data` al cargar la página.
- En el catálogo, tres funciones distintas (`cargarServicios x2` y `cargarServiciosFiltro`) competían por llenar el mismo contenedor `#ajaxServicios`, generando una condición de carrera que dejaba los filtros por categoría sin funcionar la mayoría de las veces.

Solución implementada: se reescribió el archivo eliminando el bloque duplicado, dejando una sola declaración de cada función y un único `DOMContentLoaded`. Se conservó `cargarServiciosFiltro()` como función única para el catálogo, ya que incluye el atributo `data-categoria` necesario para `activarFiltros()`.

```
document.addEventListener('DOMContentLoaded', function() {  
  initForm();  
  
  if (document.getElementById('dashboardContent')) {  
    loadDashboard();  
  }  
  
  if (document.getElementById('ajaxServicios')) {
```

```
cargarServiciosFiltro();  
}  
});
```

Figura 10. Inicialización única de los módulos del frontend.

Resultado: el formulario ahora genera una sola solicitud por envío, el dashboard hace una sola petición a su API, y los filtros de categoría funcionan de forma consistente. Adicionalmente, el archivo se redujo de 795 a 315 líneas, lo que reduce el peso transferido al navegador.

## 22.2. Ausencia de compresión en las respuestas del servidor

Descripción del problema: la auditoría de rendimiento (Lighthouse) reportó “No compression applied” para las respuestas del servidor, es decir, archivos como main.js (21 KB) y styles2.css (3.1 KB) se transfieren sin comprimir.

Solución implementada: se incorpora el middleware compression, que comprime automáticamente las respuestas con gzip/brotli según lo que soporte el navegador del cliente.

```
// index.js  
const compression = require('compression');  
app.use(compression());
```

Figura 11. Middleware de compresión de respuestas.

Resultado: reducción del tamaño de transferencia de los archivos estáticos y de las respuestas JSON de las APIs (/api/servicios, /api/dashboard/data), disminuyendo el tiempo de carga, especialmente en redes móviles.

## 22.3. Recursos que bloquean el renderizado inicial

Descripción del problema: Lighthouse identificó que bootstrap.bundle.min.js, bootstrap.min.css, main.js, styles2.css y la fuente de Google Fonts se cargan de forma bloqueante, generando una cadena de dependencias críticas de hasta 2,487 ms antes de poder pintar la página.

Solución implementada: se añadió el atributo defer al script de Bootstrap y a main.js, para que se descarguen en paralelo sin bloquear el parseo del HTML, y se agregaron etiquetas de preconnect hacia los orígenes externos utilizados.

```
<!-- partials/header.ejs -->
<link rel="preconnect" href="https://cdn.jsdelivr.net">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
defer></script>
<script src="/js/main.js" defer></script>
```

Figura 12. Carga diferida de scripts y preconexión a orígenes externos.

Resultado: la latencia de la ruta crítica de carga se reduce, ya que los scripts dejan de bloquear el primer renderizado de la página.

## 22.4. CSS de Bootstrap mayormente sin usar

Descripción del problema: de los 27.1 KiB de bootstrap.min.css cargados desde el CDN, Lighthouse estima que ~25.2 KiB no se utilizan en la página, ya que el proyecto solo aprovecha componentes básicos (grid, navbar, cards, botones).

Solución propuesta: dado que el proyecto usa pocos componentes de Bootstrap, se documenta la sustitución del bundle completo por una hoja de estilos personalizada en styles2.css que replique únicamente las clases utilizadas (.card, .btn, .navbar, sistema de grid), o el uso de los paquetes modulares bootstrap-grid y bootstrap-utilities en lugar del bundle completo.

Resultado esperado: reducción de hasta ~25 KB en la transferencia de CSS si se aplica la sustitución en una siguiente iteración del proyecto.

## 22.5. Cambio de diseño (layout shift) al cargar la fuente Inter

Descripción del problema: Lighthouse detectó un Cumulative Layout Shift (CLS) de 0.007 en elementos p.text-muted, causado porque el archivo woff2 de la fuente “Inter” (Google Fonts) tarda 2,487 ms en cargar, provocando que el texto se desplace cuando la fuente reemplaza a la fuente de respaldo del sistema.

Solución implementada: se mantiene el parámetro display=swap y se define una pila de fuentes de respaldo con proporciones similares a Inter en styles2.css, además del preconnect descrito en el inciso C) para acelerar la descarga de la fuente.

```
/* styles2.css */
body {
  font-family: 'Inter', -apple-system, 'Segoe UI', Arial, sans-serif;
}
```

Figura 13. Pila de fuentes de respaldo para reducir el salto visual.

Resultado: se reduce el salto visual perceptible al terminar de cargar la tipografía, ya que la fuente de respaldo tiene proporciones más similares a Inter.

## 22.6. JavaScript sin minificar

Descripción del problema: Lighthouse estima un ahorro de aproximadamente 4 KB si main.js se minifica antes de servirse en producción.

Solución propuesta: se documenta el uso de una herramienta de minificación (terser o esbuild) como parte del proceso de build, generando main.min.js para producción mientras se mantiene main.js legible para desarrollo.

```
npx terser public/js/main.js -o public/js/main.min.js -c -m
```

Figura 14. Comando de minificación de JavaScript para producción.

Resultado: reducción adicional del peso de transferencia de JavaScript en el entorno de producción.

## 23. Reflexión final

El desarrollo de SeguriTech representó un proceso de aprendizaje integral que va más allá de la escritura de código. A lo largo del proyecto, el equipo tuvo que tomar decisiones técnicas reales: elegir entre distintos enfoques de validación, definir cómo estructurar las consultas a la base de datos sin comprometer la seguridad, y entender por qué una arquitectura como MVC, que inicialmente puede parecer una complejidad innecesaria, facilita enormemente la detección y corrección de errores conforme el proyecto crece.

Uno de los aprendizajes más significativos fue comprender la diferencia entre un sistema que simplemente funciona y uno que funciona de forma segura. Durante el desarrollo se identificaron

vulnerabilidades concretas —XSS, inyección SQL, subida de archivos sin restricciones— y se implementaron controles específicos para cada una. Este ejercicio hizo evidente que la seguridad no es un módulo que se agrega al final del proyecto, sino una consideración que debe estar presente desde el diseño inicial de la base de datos, pasando por la definición de las rutas, hasta la forma en que las vistas renderizan los datos.

También resultó revelador el proceso de auditoría con Google Lighthouse. Problemas como el código duplicado en `main.js` —que provocaba condiciones de carrera en los filtros del catálogo y doble inserción en la base de datos— no eran visibles a simple vista durante las pruebas manuales, pero tenían consecuencias funcionales reales y demostrables. Este tipo de hallazgo refuerza la importancia de usar herramientas de análisis externas además de las pruebas propias del equipo.

Finalmente, el proyecto dejó claro que un sistema en producción requiere una capa adicional de controles que en desarrollo local pueden ignorarse: HTTPS para cifrar el tráfico, rate limiting para proteger los endpoints de solicitudes masivas, cabeceras de seguridad HTTP, y autenticación para proteger el acceso al dashboard. Estas mejoras identificadas no son fallas del proyecto actual, sino la hoja de ruta natural de un sistema que ha demostrado ser funcional y está listo para evolucionar.

## 24. Conclusiones

El proyecto SeguriTech cumplió con los objetivos planteados al inicio del desarrollo. Se diseñó e implementó una aplicación web dinámica funcional bajo la arquitectura MVC, utilizando el stack tecnológico definido: Node.js, Express, EJS, MySQL y Bootstrap. La separación entre modelos, controladores y vistas permitió mantener el código organizado, facilitar la depuración y delimitar claramente las responsabilidades de cada componente del sistema.

En cuanto a los requisitos funcionales, el sistema implementa el catálogo de servicios tecnológicos con filtrado por categoría mediante AJAX, el formulario de cotización con validación en cliente y servidor, la subida y control de archivos adjuntos, el dashboard estadístico

con indicadores dinámicos, y la persistencia completa de datos en una base de datos relacional con tres tablas relacionadas mediante llaves primarias y foráneas.

Respecto a la seguridad, se implementaron ocho mecanismos funcionales y demostrables: prevención de inyección SQL mediante consultas parametrizadas, prevención de XSS mediante escape automático en EJS y sanitización con la librería validator, validación de datos de entrada en servidor independiente del cliente, restricción de archivos subidos por extensión y tipo MIME, gestión de credenciales mediante variables de entorno, manejo centralizado de errores sin exposición de información interna, política CORS explícita e integridad referencial a nivel de motor de base de datos. Adicionalmente se documentó el modelo de amenazas bajo la metodología STRIDE, identificando tanto las amenazas mitigadas como aquellas pendientes para una siguiente iteración.

El proyecto también evidenció que el desarrollo de software profesional implica ciclos de revisión y corrección continuos. Problemas detectados durante la auditoría —código duplicado, recursos bloqueantes, ausencia de compresión— fueron analizados, documentados y resueltos, demostrando la capacidad del equipo para identificar causas raíz y aplicar soluciones técnicas justificadas.

Como trabajo futuro, las prioridades identificadas son la implementación de HTTPS para cifrado del tráfico en producción, autenticación con roles de usuario para proteger el acceso al dashboard, rate limiting en los endpoints críticos, y la incorporación de cabeceras de seguridad HTTP mediante Helmet. Estas mejoras elevarían el sistema de un nivel de seguridad adecuado para desarrollo local a uno apropiado para un entorno de producción real.

## 25. Referencias APA

- *Express/Node introduction - Learn web development* | MDN. (2025, September 11). [https://developer.mozilla.org/en-US/docs/Learn\\_web\\_development/Extensions/Server-side/Express\\_Nodejs/Introduction](https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs/Introduction)
- Copes, F. (2024, October 4). *The Express + Node.js handbook – Learn the Express JavaScript Framework for beginners*. freeCodeCamp.org. <https://www.freecodecamp.org/news/the-express-handbook/>

- *Introducción a Express/Node - Aprende desarrollo web* | MDN. (n.d.).  
[https://developer.mozilla.org/es/docs/Learn\\_web\\_development/Extensions/Server-side/Express\\_Nodejs/Introduction](https://developer.mozilla.org/es/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs/Introduction)
- *Referencia de Elementos HTML - HTML* | MDN. (n.d.).  
<https://developer.mozilla.org/es/docs/Web/HTML/Reference/Elements>
- *CSS properties - CSS* | MDN. (2026, March 25).  
<https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties>
- *JavaScript reference - JavaScript* | MDN. (2026, May 22).  
<https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference>